

Module 7 Lab:

SonarQube Tool – Logic-Based Coverage

JR Clark (002035589)

NEU CSYE6255 (Summer 2026)

Professor Omar Badreldin / TA Karthik Kashyap

Table of Contents

<i>PART 1: Tool Installation & Project Setup</i>	<i>4</i>
<i>PART 2: Static Analysis.....</i>	<i>4</i>
<i>PART 3: Code Quality Assessment & Vulnerability Identification</i>	<i>5</i>
<i>PART 4: Critical Evaluation & Analysis</i>	<i>6</i>
<i>APPENDIX 1: Tool Installation & Project Setup.....</i>	<i>8</i>
Figure 1. Install Docker.app.....	8
Figure 2. Confirmation of Docker Desktop installation & connection to Github	8
Figure 3. Confirming Docker in Terminal.....	8
Figure 4. Checking for sonar-scanner in Terminal.....	9
Figure 5. Confirming installation of sonar-scanner in Terminal with “sonar-scanner –version”	9
Figure 6. Setting up a SonarQube Community Container with “Gordon”	10
Figure 7. Docker confirms setup of Container	10
Figure 8. Successful Localhost Login – SonarQube Container	11
Figure 9. Confirm Cloned Repositories for SonarQube Trial	11
Figure 10. Final confirmation of all tools installed in Terminal	12
<i>APPENDIX 2: Project Setup for Apache commons-lang.....</i>	<i>13</i>
Figure 11. Token and Scanner code generation for commons-lang.....	13
Figure 12. Compile commons-lang with “mvn clean compile” from root.....	14
Figure 13. Create sonar-project.properties at commons-lang project root per Karthik’s instructions (section 23):	14
Figure 14. Comparison of Instructions against saved sonar-project.properties file	15
Figure 15. Run sonar-scanner on commons-lang.....	15
Figure 16. Build success message for sonar-scanner on commons-lang.....	15
Figure 17. SonarQube Dashboard Confirmation	16
<i>APPENDIX 3: Project Setup for Django.....</i>	<i>17</i>
Figure 18. Token and Scanner code generation for django	17
Figure 19. Installation confirmation for pysonar.....	18
Figure 20. Scan Django using commands provided in SonarQube	18
Figure 21. Confirmation of completion of scan on Django	19
Figure 22. SonarQube Dashboard Confirmation	19
<i>APPENDIX 4: CodeMR setup for commons-lang</i>	<i>20</i>
Figure 23. Open Eclipse & import commons-lang to run CodeMR	20
<i>APPENDIX 5: Project Setup for spring-framework</i>	<i>21</i>
Figure 24. Token and Scanner code generation for spring-framework.....	21
Figure 25. Use Brew to install Gradle.....	22
Figure 26. Update build.gradle in spring-framework project to include older version of SonarQube plugin... ..	22
Figure 27. Troubleshooting compatibility issues with Gradle (with Claude.ai)	23
Figure 28. Installing older version of Java (21) to help resolve Gradle/SonarQube issue	24
Figure 29. Gradle.properties now points to older version of Java for spring-framework	24
Figure 30. Down versioning SonarQube for compatibility	24
Figure 31. Terminal confirmation of successful SonarQube build for spring-framework	25
Figure 32. Localhost web SonarQube dashboard for spring-framework	25
<i>APPENDIX 6: SonarQube Comparative Summaries</i>	<i>26</i>
Table 1: SonarQube Results Overview	26

<i>APPENDIX 7: CodeMR Report Comparative Summaries</i>	<i>27</i>
Table 2: Project Metrics Summary	27
Table 3: Project “C” Metrics Summary.....	27
<i>APPENDIX 8: CodeMR Report Visuals Comparison</i>	<i>28</i>
Figure 33. Analysis & Distribution of Quality Attributes	28
Figure 35. Metric Values in Package.....	29
Figure 36. Metric Values in Treemap Chart.....	30
Figure 37. Package Dependencies.....	31
<i>REFERENCES</i>	<i>32</i>
<i>Methodology and Use of AI Tools</i>	<i>33</i>

PART 1: Tool Installation & Project Setup

Tools installed following the Lab 7 instructions (Appendix 1, Figures 1–10): Docker Desktop, SonarQube Community Edition via Docker container, SonarScanner CLI via Homebrew, pysonar for Python analysis, Gradle via Homebrew, Java 21 via Homebrew (required for spring-framework compatibility). CodeMR and Eclipse IDE were previously installed for earlier labs and used to analyze commons-lang for comparison with JavaHuskies_Final (see Appendix 8).

Three projects selected spanning two languages: **Django** (Python, 123k LOC), a production-grade web framework; **Apache commons-lang** (Java, 33k LOC), a widely-used open-source utility library; **Spring Framework** (Java, 410k LOC), a large-scale enterprise Java framework (which was added when commons-lang fell below the < 100k LOC requirement). I have since learned that there are Chrome browser extensions that can help estimate repo size.

PART 2: Static Analysis

SonarQube was deployed locally in client-server configuration: SonarQube Community Edition in Docker container as server, SonarScanner CLI and pysonar as clients. **Commons-lang** required Maven compilation prior to scanning and a sonar-project.properties configuration file (Appendix 2, Figures 11–17). **Django** was scanned via pysonar using token-authenticated commands generated by the SonarQube UI (Appendix 3, Figures 18–22). **Spring Framework** required significant compatibility troubleshooting before successful scan. including Gradle plugin versioning, Java 25/Gradle version conflicts, and resolution via SonarQube Gradle plugin version 7.3.1.8318 (Appendix 5, Figures 24–32). Full results in Table 1 (Appendix 6). CodeMR analysis conducted on commons-lang via Eclipse (Appendix 4, Figure 23). During execution,

SonarScanner loads quality profiles, indexes all the files in the repo, runs language sensors and uploads the report (see Figures 15-16 for partial display of actions printed in Terminal).

PART 3: Code Quality Assessment & Vulnerability Identification

All three projects passed the SonarQube Quality Gate despite open issues: a passing gate reflects configured policy thresholds, not zero defects. Coverage was 0.0% across all projects as no test reports were imported; SonarQube does not execute tests independently. Scale differences across the three projects are significant: **spring-framework**'s 858 reliability issues and 16k maintainability issues reflect its size and multi-module complexity rather than poor engineering. **Django**'s 74 security hotspots relative to 12 confirmed security issues reflects its role as a web framework with significant security-sensitive surface area, including authentication, sessions, and HTTP handling, each of which requires human review rather than automated confirmation. **commons-lang**'s zero security issues and strong coupling metrics reflect its focused, well-isolated utility library design.

Blocker-level issues for **commons-lang** are largely consistency errors – field/methods naming conflicts and null return type mismatches -- while **Django** and **spring-framework** both carry “blocker” vulnerabilities including cross-site scripting, SQL injection, Denial of Service (DoS) and weak cryptography (see OpenLogic and CVEDetails, for independent corroboration).

A key takeaway is that these are actively maintained, widely-deployed codebases, yet SonarQube identifies current unresolved security issues. Organizations that consume open-source libraries may patch vulnerabilities in their own implementations without contributing fixes upstream, leaving the underlying repository and other downstream consumers exposed. CVEDetails search results for “Spring Framework” shows four different versions (VMware, Pivotal Software, Springsource and Pivotal). The same framework is being maintained as

separate versions with separate known vulnerabilities. For open-source repos, the gap between vulnerability identification and production patching across the ecosystem can span months or years. This type of gap underscores why integrating static analysis into CI/CD pipelines matters: catching issues before they ship is much better than coordinating downstream remediation after public disclosure (see the Log4Shell issue, Wikipedia 2022). CodeMR analysis of **commons-lang** is summarized in Appendix 7 (Tables 2–3) and Appendix 8 (Figures 33–37), including a comparative analysis against **JavaHuskies Final**.

Recommendations: **spring-framework**'s security issues may warrant priority review if you are using the open-source repo, but if you are using VMWare, for example, those may already be remediated; **Django**'s ratio suggests manual review backlog to resolve maintainability issues; **commons-lang**'s 46 problematic classes are candidates for targeted refactoring.

PART 4: Critical Evaluation & Analysis

SonarQube and CodeMR are complementary tools serving different analytical purposes. SonarQube excels at multi-language support, CI/CD integration, longitudinal tracking, and security vulnerability identification. Its New Code vs. Overall Code view reflects its primary use case as a continuous monitoring tool. Limitations include 0% coverage without test report import, no structural visualization, and pre-scan compilation of Java binaries. The **spring-framework** troubleshooting sequence (Appendix 5) illustrates a real-world practitioner reality: compatibility matrices between analysis tools, build systems, and Java versions introduce friction that must be anticipated at the pipeline configuration level rather than resolved per-scan. In production environments, these compatibility issues would be resolved once at the pipeline configuration level.

CodeMR is best suited for early architectural review on small-to-medium Java projects where visual communication of quality is a priority, complementing rather than replacing tools like SonarQube for security analysis, coverage integration, and CI/CD enforcement.

Likewise, SonarQube is not a one-time diagnostic but a continuous practice. The value of static analysis compounds over time: a single snapshot identifies issues, but integrating both tools into a CI/CD pipeline means running analysis against known-good configurations, tracking changes across releases, and surfacing critical issues before they reach production. Pinning plugin versions, JDK versions, and build tool configurations at the pipeline level ensures consistent, repeatable results. The combination of SonarQube for multi-language security monitoring and longitudinal quality tracking, with CodeMR for architectural visualization and pre-refactoring analysis, provides complementary coverage across the software quality spectrum. Prioritization of identified issues remains a human responsibility: both tools surface candidates for review, but the decision about what to fix, when, and in which version belongs to the engineering team.

APPENDIX 1: Tool Installation & Project Setup

Figure 1. Install Docker.app

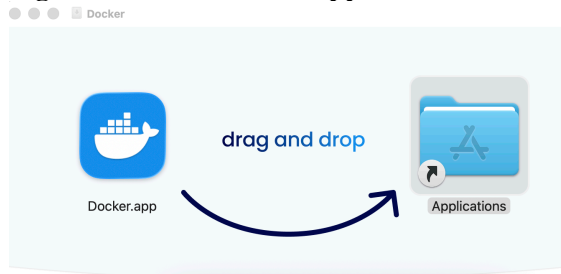


Figure 2. Confirmation of Docker Desktop installation & connection to Github

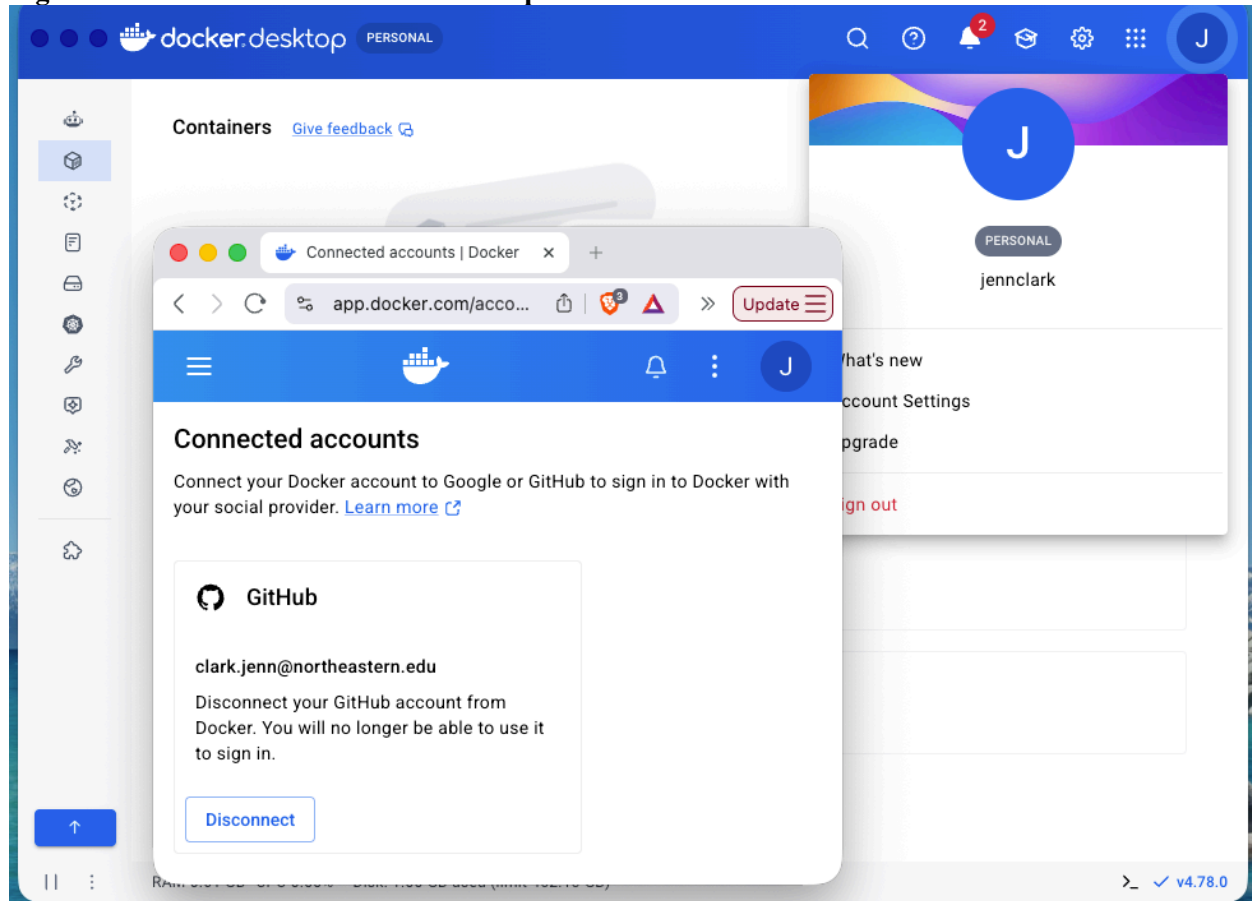


Figure 3. Confirming Docker in Terminal

```
blackcatranch-m3 — -zsh — 80x24

Last login: Thu Jun 11 18:23:17 on ttys000
blackcatranch-m3@JRC-M3 ~ % docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
blackcatranch-m3@JRC-M3 ~ %
```

Figure 4. Checking for sonar-scanner in Terminal

```
[blackcatranch-m3@JRC-M3 ~ % sonar-scanner --version ]
zsh: command not found: sonar-scanner
[blackcatranch-m3@JRC-M3 ~ % brew install sonar-scanner ]
==> Auto-updating Homebrew...
Adjust how often this is run with `HOMEBREW_AUTO_UPDATE_SECS` or disable with
`HOMEBREW_NO_AUTO_UPDATE=1`. Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (
see `man brew`).
✓ JSON API packages.arm64_tahoe.jws.json           Downloaded 15.2MB/ 15.2MB
==> Auto-updated Homebrew!
Updated 2 taps (homebrew/core and homebrew/cask).
==> New Formulae
asccli: App Store Connect CLI to manage apps, versions, and screenshots
avisynthplus: Improved version of the AviSynth frameserver
beets: Music library manager and tagger
btft: CLI tool for datetime arithmetic, parsing, formatting and more
buffa: Pure-Rust Protocol Buffers implementation with editions support
cargo-show-asm: Show assembly, LLVM-IR, MIR, and WASM generated for Rust code
concord: Terminal user interface client for Discord
container-use: Dev envs for coding agents. Run multiple agents safely with your
stack
container2wasm: Container to WASM converter
cyclonedx-cli: Tool for analysis and manipulation of CycloneDX SBOMs
dbx-cli: Command-line interface for DBX database connections, schema, and safe q
ueries
```

Figure 5. Confirming installation of sonar-scanner in Terminal with “sonar-scanner –version”

```
blackcatranch-m3 — zsh — 80x24
fontconfig
graphite2
harfbuzz
==> Do you want to proceed with the installation? [y/n]
==> Fetching downloads for: sonar-scanner
✓ Bottle Manifest fontconfig (2.18.1)           Downloaded 13.8KB/ 13.8KB
✓ Bottle Manifest graphite2 (1.3.15)            Downloaded 8.0KB/ 8.0KB
✓ Bottle Manifest harfbuzz (14.2.1)             Downloaded 33.9KB/ 33.9KB
✓ Bottle fontconfig (2.18.1)                   Downloaded 417.0KB/417.0KB
✓ Bottle graphite2 (1.3.15)                    Downloaded 92.4KB/ 92.4KB
✓ Bottle harfbuzz (14.2.1)                     Downloaded 3.5MB/ 3.5MB
✓ Bottle sonar-scanner (8.1.0.6389)            Downloaded 9.2MB/ 9.2MB
==> Pouring sonar-scanner--8.1.0.6389.all.bottle.tar.gz
📦 /opt/homebrew/Cellar/sonar-scanner/8.1.0.6389: 9 files, 10.8MB
==> Running `brew cleanup sonar-scanner`...
Disable this behaviour by setting `HOMEBREW_NO_INSTALL_CLEANUP=1`.
Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
[blackcatranch-m3@JRC-M3 ~ % sonar-scanner --version ]
16:17:56.886 INFO Scanner configuration file: /opt/homebrew/Cellar/sonar-scanner/8.1.0.6389/libexec/conf/sonar-scanner.properties
16:17:56.887 INFO Project root configuration file: NONE
16:17:56.895 INFO SonarScanner CLI 8.1.0.6389
16:17:56.896 INFO Mac OS X 26.5 aarch64
blackcatranch-m3@JRC-M3 ~ %
```

Figure 6. Setting up a SonarQube Community Container with “Gordon”

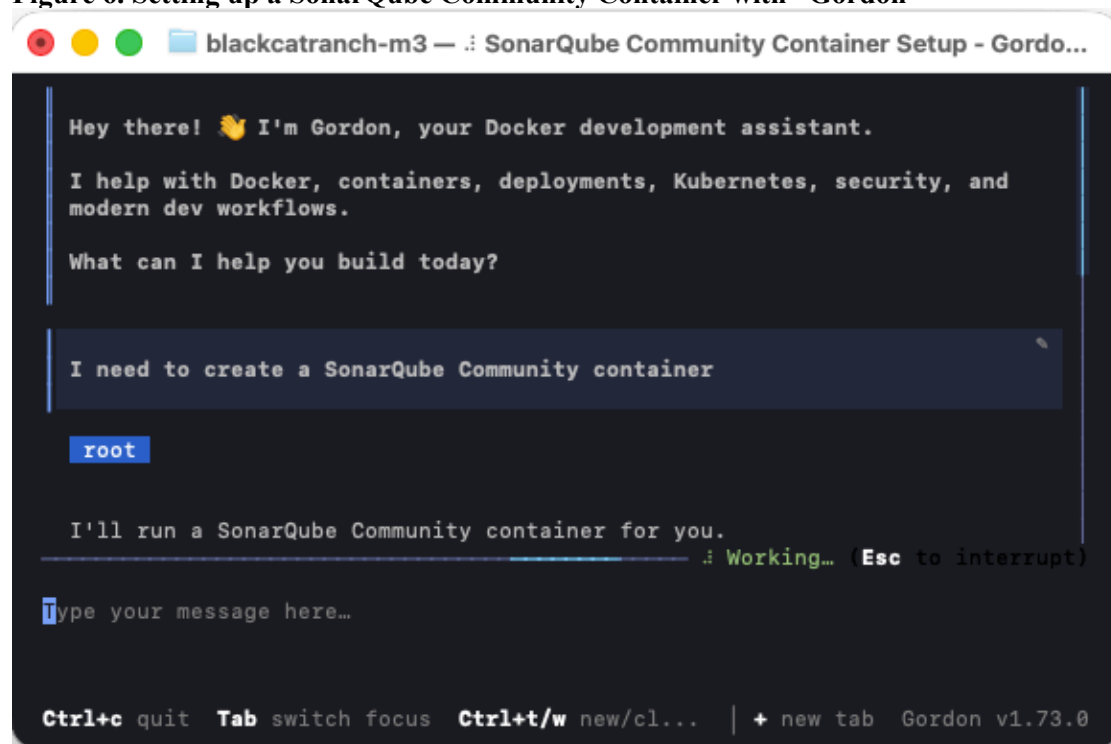


Figure 7. Docker confirms setup of Container

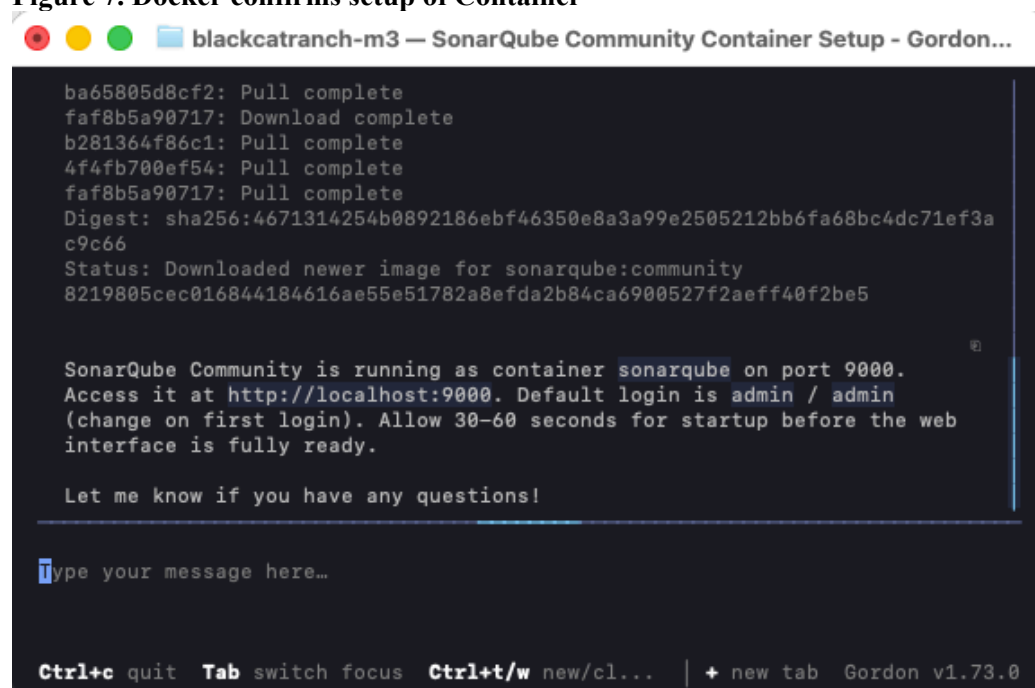


Figure 8. Successful Localhost Login – SonarQube Container

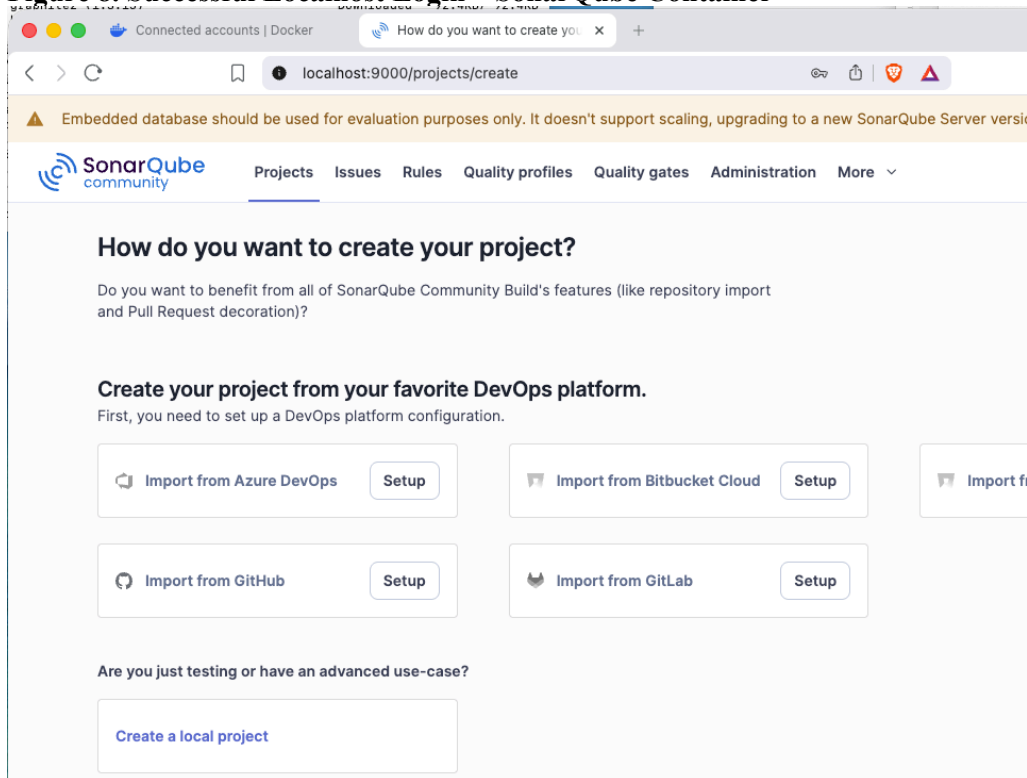


Figure 9. Confirm Cloned Repositories for SonarQube Trial

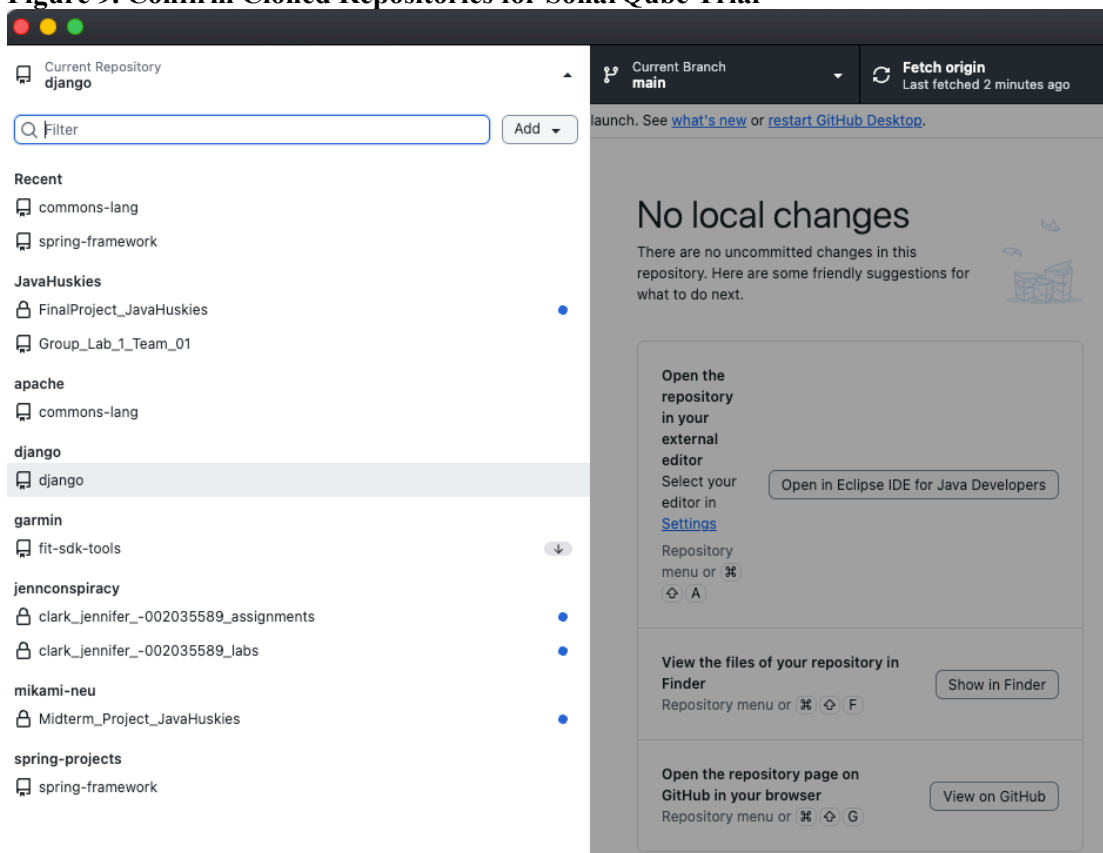


Figure 10. Final confirmation of all tools installed in Terminal

```
blackcatranch-m3 — zsh — 150x25

[blackcatranch-m3@JRC-M3 ~ % git --version; docker --version; docker ps; java -version; brew --version; sonar-scanner --version
git version 2.52.0
Docker version 29.5.3, build d1c06ef
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS                               NAMES
8219805cec01   sonarqube:community "/opt/sonarqube/dock..." 11 minutes ago Up 11 minutes  0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp  sonarqube
openjdk version "25.0.3" 2026-04-21 LTS
OpenJDK Runtime Environment Temurin-25.0.3+9 (build 25.0.3+9-LTS)
OpenJDK 64-Bit Server VM Temurin-25.0.3+9 (build 25.0.3+9-LTS, mixed mode, sharing)
Homebrew 6.0.2
16:32:11.840 INFO Scanner configuration file: /opt/homebrew/Cellar/sonar-scanner/8.1.0.6389/libexec/conf/sonar-scanner.properties
16:32:11.841 INFO Project root configuration file: NONE
16:32:11.848 INFO SonarScanner CLI 8.1.0.6389
16:32:11.850 INFO Mac OS X 26.5 aarch64
blackcatranch-m3@JRC-M3 ~ %
```

APPENDIX 2: Project Setup for Apache commons-lang

Figure 11. Token and Scanner code generation for commons-lang

The screenshot shows the SonarQube Community web interface. The browser address bar indicates the URL: `localhost:9000/tutorials?id=commons-lang&selectedTutorial=local`. A warning banner at the top states: "Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine." The navigation bar includes links for Projects, Issues, Rules, Quality profiles, Quality gates, Administration, and More. The main content area is titled "Analyze your project" and includes a sub-header "commons-lang / Analysis Method / Locally". Below this, a message states: "We initialized your project on SonarQube Community Build, now it's up to you to launch analyses!". The interface is divided into two main steps: 1. Provide a token, and 2. Run analysis on your project. Step 1 shows a green checkmark and the token: `sqp_cf821ebf3a383217f55aa30db9fc98d4b8c583f3`. Step 2 asks "What option best describes your project?" and provides buttons for Maven, Gradle, JS/TS & Web, .NET, Python, and Other (for Go, PHP, ...). Below this, the section "Execute the Scanner for Maven" provides instructions: "Running a SonarQube analysis with Maven is straightforward. You just need to run the following command in your project's folder." A code block contains the command: `mvn clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \n-Dsonar.projectKey=commons-lang \n-Dsonar.projectName='commons-lang' \n-Dsonar.host.url=http://localhost:9000 \n-Dsonar.token=sqp_cf821ebf3a383217f55aa30db9fc98d4b8c583f3`. A "Copy" button is next to the code block.

commons-lang / Analysis Method / Locally

Analyze your project

We initialized your project on SonarQube Community Build, now it's up to you to launch analyses!

1 Provide a token

✓ Analyze "commons-lang": `sqp_cf821ebf3a383217f55aa30db9fc98d4b8c583f3`

2 Run analysis on your project

What option best describes your project?

Maven Gradle JS/TS & Web .NET Python Other (for Go, PHP, ...)

Execute the Scanner for Maven

Running a SonarQube analysis with Maven is straightforward. You just need to run the following command in your project's folder.

```
mvn clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \n-Dsonar.projectKey=commons-lang \n-Dsonar.projectName='commons-lang' \n-Dsonar.host.url=http://localhost:9000 \n-Dsonar.token=sqp_cf821ebf3a383217f55aa30db9fc98d4b8c583f3
```

Copy

Figure 12. Compile commons-lang with “mvn clean compile” from root

```
commons-lang -- zsh -- 139x39

Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-build-api/1.2.0/plexus-build-api-1.2.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-build-api/1.2.0/plexus-build-api-1.2.0.pom (4.7 kB at 315 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-interpolation/1.29/plexus-interpolation-1.29.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-interpolation/1.29/plexus-interpolation-1.29.pom (4.7 kB at 295 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-filtering/3.5.0/maven-filtering-3.5.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-filtering/3.5.0/maven-filtering-3.5.0.jar (61 kB at 4.0 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-build-api/1.2.0/plexus-build-api-1.2.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-build-api/1.2.0/plexus-build-api-1.2.0.jar (88 kB at 5.9 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-interpolation/1.29/plexus-interpolation-1.29.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-interpolation/1.29/plexus-interpolation-1.29.jar (88 kB at 5.9 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-build-api/1.2.0/plexus-build-api-1.2.0.jar (6.6 kB at 253 kB/s)
[INFO] skip non existing resourceDirectory /Users/blackcatranch-m3/Documents/GitHub/commons-lang/src/main/resources
[INFO] Copying 2 resources from . to target/classes/META-INF
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ commons-lang3 ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 263 source files with javac [debug release 8] to target/classes
[WARNING] source value 8 is obsolete and will be removed in a future release
[WARNING] target value 8 is obsolete and will be removed in a future release
[WARNING] To suppress warnings about obsolete options, use -Xlint:-options.
[INFO] /Users/blackcatranch-m3/Documents/GitHub/commons-lang/src/main/java/org/apache/commons/lang3/builder/ToStringStyle.java: Some input files use or override a deprecated API.
[INFO] /Users/blackcatranch-m3/Documents/GitHub/commons-lang/src/main/java/org/apache/commons/lang3/builder/ToStringStyle.java: Recompile with -Xlint:deprecation for details.
[INFO] /Users/blackcatranch-m3/Documents/GitHub/commons-lang/src/main/java/org/apache/commons/lang3/AppendableJoiner.java: Some input files use unchecked or unsafe operations.
[INFO] /Users/blackcatranch-m3/Documents/GitHub/commons-lang/src/main/java/org/apache/commons/lang3/AppendableJoiner.java: Recompile with -Xlint:unchecked for details.
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 14.600 s
[INFO] Finished at: 2026-06-19T16:55:50-07:00
[INFO]
blackcatranch-m3@JRC-M3 commons-lang %
```

Figure 13. Create sonar-project.properties at commons-lang project root per Karthik’s instructions (section 23):

```
commons-lang -- zsh -- 139x39

blackcatranch-m3@JRC-M3 commons-lang % cat > sonar-project.properties <<EOF
sonar.projectKey=commons-lang
sonar.projectName=commons-lang
sonar.sources=src/main/java
sonar.java.binaries=target/classes
sonar.host.url=http://localhost:9000
EOF
blackcatranch-m3@JRC-M3 commons-lang % ls -la
total 520
drwxr-xr-x@ 19 blackcatranch-m3  staff   608 Jun 19 17:04 .
drwxr-xr-x@ 11 blackcatranch-m3  staff   352 Jun 19 15:41 ..
-rw-r--r--@  1 blackcatranch-m3  staff  1423 Jun 19 09:39 .asf.yaml
-rw-r--r--@  1 blackcatranch-m3  staff   6148 Jun 19 15:44 .DS_Store
drwxr-xr-x@ 14 blackcatranch-m3  staff   448 Jun 19 15:42 .git
-rw-r--r--@  1 blackcatranch-m3  staff   950 Jun 19 09:39 .gitattributes
drwxr-xr-x@  6 blackcatranch-m3  staff   192 Jun 19 09:39 .github
-rw-r--r--@  1 blackcatranch-m3  staff   318 Jun 19 09:39 .gitignore
-rw-r--r--@  1 blackcatranch-m3  staff   930 Jun 19 09:39 CODE_OF_CONDUCT.md
-rw-r--r--@  1 blackcatranch-m3  staff   7188 Jun 19 09:39 CONTRIBUTING.md
-rw-r--r--@  1 blackcatranch-m3  staff  11359 Jun 19 09:39 LICENSE.txt
-rw-r--r--@  1 blackcatranch-m3  staff    174 Jun 19 09:39 NOTICE.txt
-rw-r--r--@  1 blackcatranch-m3  staff  35274 Jun 19 09:39 pom.xml
-rw-r--r--@  1 blackcatranch-m3  staff   6964 Jun 19 09:39 README.md
-rw-r--r--@  1 blackcatranch-m3  staff  160766 Jun 19 09:39 RELEASE-NOTES.txt
-rw-r--r--@  1 blackcatranch-m3  staff    901 Jun 19 09:39 SECURITY.md
-rw-r--r--@  1 blackcatranch-m3  staff    161 Jun 19 17:04 sonar-project.properties
drwxr-xr-x@ 10 blackcatranch-m3  staff   320 Jun 19 15:41 src
drwxr-xr-x@  8 blackcatranch-m3  staff   256 Jun 19 16:55 target
blackcatranch-m3@JRC-M3 commons-lang %
```

Figure 14. Comparison of Instructions against saved sonar-project.properties file

A side-by-side comparison of two sonar-project.properties files. The left file, titled 'Configuration:', contains settings for 'InventorySystem'. The right file, titled 'commons-lang — nano sonar-project', contains settings for 'commons-lang'.

```
Configuration:
sonar.projectKey=InventorySystem
sonar.projectName=InventorySystem
sonar.sources=src
sonar.java.binaries=target/classes
sonar.host.url=http://localhost:9000

UW PICO 5.09
sonar.projectKey=commons-lang
sonar.projectName=commons-lang
sonar.sources=src/main/java
sonar.java.binaries=target/classes
sonar.host.url=http://localhost:9000
```

Figure 15. Run sonar-scanner on commons-lang

A terminal window showing the execution of the sonar-scanner command. The output includes various informational messages about the scanner configuration, project root, and the start of the SonarScanner engine.

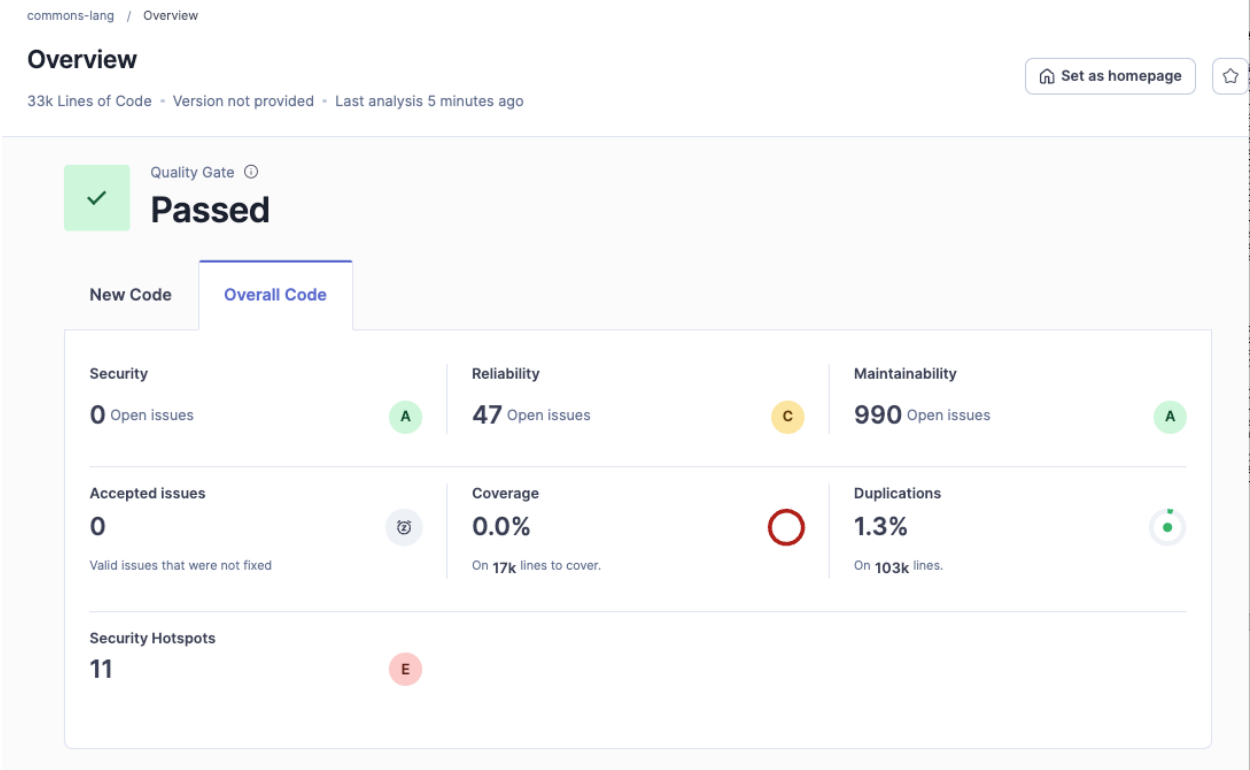
```
commons-lang — -zsh — 145x66
blackcatranch-m3@JRC-M3 commons-lang % sonar-scanner -Dsonar.token=sqp_cf821ebf3a383217f55aa30db9fc98d4b8c583f3
17:07:50.741 INFO Scanner configuration file: /opt/homebrew/Cellar/sonar-scanner/8.1.0.6389/libexec/conf/sonar-scanner.properties
17:07:50.750 INFO Project root configuration file: /Users/blackcatranch-m3/Documents/GitHub/commons-lang/sonar-project.properties
17:07:50.750 INFO SonarScanner CLI 8.1.0.6389
17:07:50.751 INFO Mac OS X 26.5 aarch64
17:07:53.587 INFO Communicating with SonarQube Community Build 26.6.0.123539
17:07:53.587 INFO JRE provisioning: os[macos], arch[arm64]
17:07:57.433 INFO Starting SonarScanner Engine...
17:07:57.433 INFO Java 21.0.9 Eclipse Adoptium (64-bit)
17:07:59.680 INFO Load global settings
17:07:59.897 INFO Load global settings (done) | time=218ms
17:07:59.899 INFO Server id: 147B411E-AZ7iL8YsREWy8I8GwEb0
17:07:59.903 INFO Loading required plugins
17:07:59.903 INFO Load plugins index
17:08:00.140 INFO Load plugins index (done) | time=236ms
17:08:00.140 INFO Load/download plugins
17:08:00.416 INFO Load/download plugins (done) | time=67ms
```

Figure 16. Build success message for sonar-scanner on commons-lang

A terminal window showing the final output of the sonar-scanner command, including the analysis of source files, sensor execution, and the final success message.

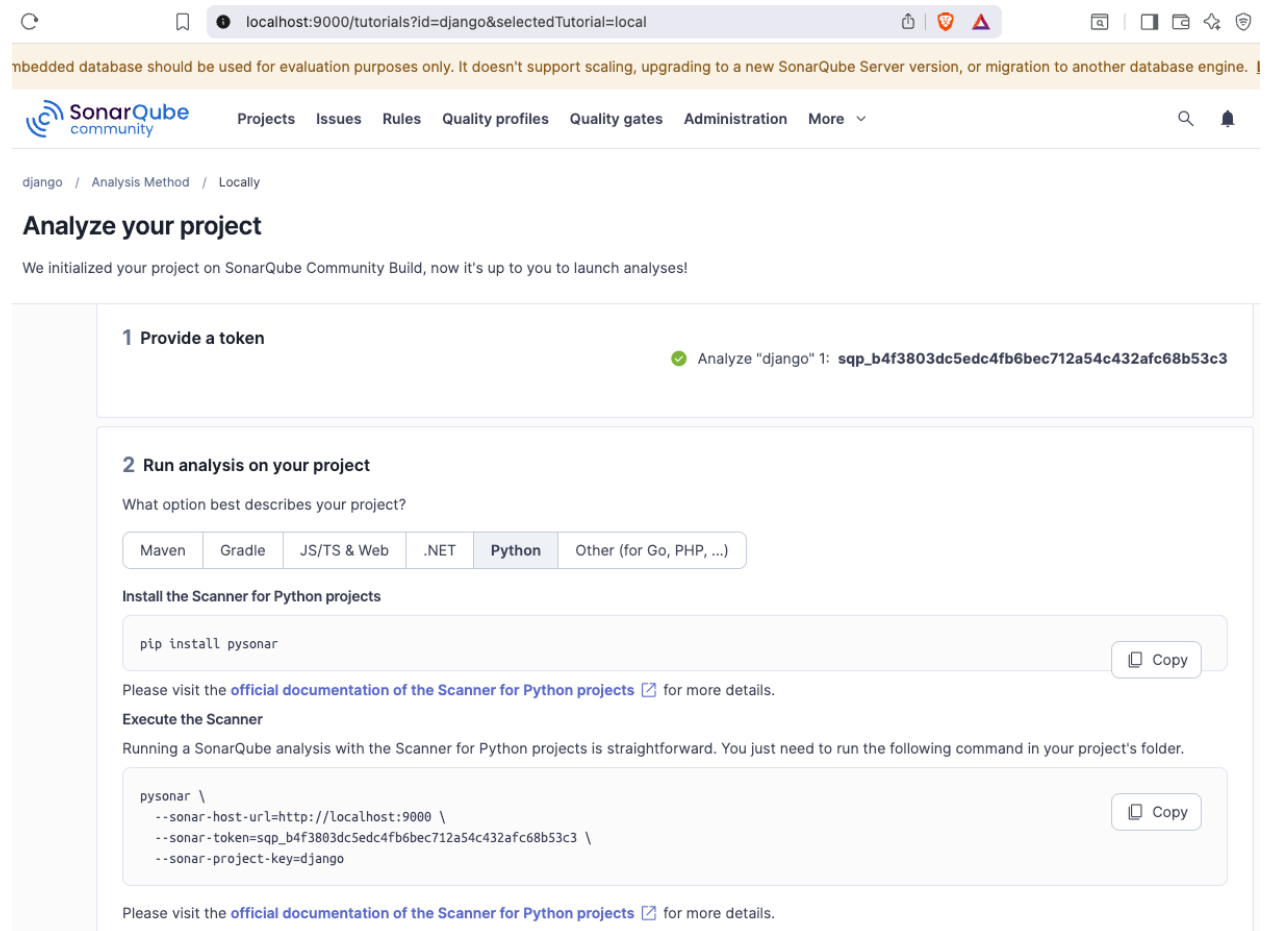
```
17:08:15.758 INFO 263 source files to be analyzed for the text and secrets analysis
17:08:16.163 INFO 263/263 source files have been analyzed for the text and secrets analysis
17:08:16.167 INFO Sensor TextAndSecretsSensor [text] (done) | time=851ms
17:08:16.173 INFO ----- Run sensors on project
17:08:16.234 INFO Sensor JaCoCo Aggregate XML Report Importer [jacoco]
17:08:16.234 INFO Sensor JaCoCo Aggregate XML Report Importer [jacoco] (done) | time=0ms
17:08:16.234 INFO Sensor JavaProjectSensor [java]
17:08:16.235 INFO Sensor JavaProjectSensor [java] (done) | time=1ms
17:08:16.235 INFO Sensor Zero Coverage Sensor
17:08:16.272 INFO Sensor Zero Coverage Sensor (done) | time=37ms
17:08:16.273 INFO Sensor Java CPD Block Indexer
17:08:16.456 INFO Sensor Java CPD Block Indexer (done) | time=184ms
17:08:16.457 INFO ----- Gather SCA dependencies on project
17:08:16.459 INFO Dependency analysis skipped
17:08:16.460 INFO SCM Publisher SCM provider for this project is: git
17:08:16.461 INFO SCM Publisher 263 source files to be analyzed
17:08:21.594 INFO SCM Publisher 263/263 source files have been analyzed (done) | time=5131ms
17:08:21.609 INFO CPD Executor 85 files had no CPD blocks
17:08:21.609 INFO CPD Executor Calculating CPD for 178 files
17:08:21.648 INFO CPD Executor CPD calculation finished (done) | time=37ms
17:08:21.663 INFO SCM revision ID '034fea0a8d8b87cd3bacdf52334186d3ec74aa97'
17:08:21.794 INFO Analysis report generated in 126ms, dir size=6.8 MB
17:08:22.404 INFO Analysis report compressed in 610ms, zip size=2.1 MB
17:08:22.901 INFO Analysis report uploaded in 496ms
17:08:22.903 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/dashboard?id=commons-lang
17:08:22.903 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
17:08:22.903 INFO More about the report processing at http://localhost:9000/api/ce/task?id=83bab809-6965-443b-b083-90111f0d831f
17:08:23.107 INFO Analysis total time: 22.461 s
17:08:23.108 INFO SonarScanner Engine completed successfully
17:08:23.175 INFO EXECUTION SUCCESS
17:08:23.182 INFO Total time: 32.437s
blackcatranch-m3@JRC-M3 commons-lang %
```

Figure 17. SonarQube Dashboard Confirmation



APPENDIX 3: Project Setup for Django

Figure 18. Token and Scanner code generation for django



localhost:9000/tutorials?id=django&selectedTutorial=local

embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. !

SonarQube community Projects Issues Rules Quality profiles Quality gates Administration More ▾

django / Analysis Method / Locally

Analyze your project

We initialized your project on SonarQube Community Build, now it's up to you to launch analyses!

1 Provide a token

✔ Analyze "django" 1: **sqp_b4f3803dc5edc4fb6bec712a54c432afc68b53c3**

2 Run analysis on your project

What option best describes your project?

Maven Gradle JS/TS & Web .NET **Python** Other (for Go, PHP, ...)

Install the Scanner for Python projects

```
pip install pysonar
```

[Copy](#)

Please visit the [official documentation of the Scanner for Python projects](#) for more details.

Execute the Scanner

Running a SonarQube analysis with the Scanner for Python projects is straightforward. You just need to run the following command in your project's folder.

```
pysonar \
  --sonar-host-url=http://localhost:9000 \
  --sonar-token=sqp_b4f3803dc5edc4fb6bec712a54c432afc68b53c3 \
  --sonar-project-key=django
```

[Copy](#)

Please visit the [official documentation of the Scanner for Python projects](#) for more details.

Figure 19. Installation confirmation for pysonar

```
django — -zsh — 139x39

blackcatranch-m3@JRC-M3 django % pip install pysonar
WARNING: Cache entry deserialization failed, entry ignored
Collecting pysonar
  Downloading pysonar-1.6.0.4905-py3-none-any.whl.metadata (6.8 kB)
Collecting jproperties<3,>=2.1 (from pysonar)
  Downloading jproperties-2.1.2-py2.py3-none-any.whl.metadata (8.7 kB)
Collecting requests<3,>=2.32 (from pysonar)
  Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting tomli<3,>=2.0 (from pysonar)
  Downloading tomli-2.4.1-cp314-cp314-macosx_11_0_arm64.whl.metadata (10 kB)
Collecting six==1.13 (from jproperties<3,>=2.1->pysonar)
  Downloading six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Collecting charset_normalizer<4,>=2 (from requests<3,>=2.32->pysonar)
  Downloading charset_normalizer-3.4.7-cp314-cp314-macosx_10_15_universal2.whl.metadata (40 kB)
Requirement already satisfied: idna<4,>=2.5 in /Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/site-packages (from requests<3,>=2.32->pysonar) (3.18)
Requirement already satisfied: urllib3<3,>=1.26 in /Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/site-packages (from requests<3,>=2.32->pysonar) (2.7.0)
Requirement already satisfied: certifi==2023.5.7 in /Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/site-packages (from requests<3,>=2.32->pysonar) (2026.5.20)
Downloading pysonar-1.6.0.4905-py3-none-any.whl (46 kB)
Downloading jproperties-2.1.2-py2.py3-none-any.whl (17 kB)
Downloading requests-2.34.2-py3-none-any.whl (73 kB)
Downloading charset_normalizer-3.4.7-cp314-cp314-macosx_10_15_universal2.whl (309 kB)
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloading tomli-2.4.1-cp314-cp314-macosx_11_0_arm64.whl (149 kB)
Installing collected packages: tomli, six, charset_normalizer, requests, jproperties, pysonar
Successfully installed charset_normalizer-3.4.7 jproperties-2.1.2 pysonar-1.6.0.4905 requests-2.34.2 six-1.17.0 tomli-2.4.1
blackcatranch-m3@JRC-M3 django %
```

Figure 20. Scan Django using commands provided in SonarQube

```
django — -zsh — 116x55

blackcatranch-m3@JRC-M3 django % pysonar \
--sonar-host-url=http://localhost:9000 \
--sonar-token=sqp_b4f3803dc5edc4fb6bec712a54c432afc68b53c3 \
[ --sonar-project-key=django
INFO: Starting the analysis...
INFO: Starting SonarScanner Engine...
INFO: Java 21.0.9 Eclipse Adoptium (64-bit)
INFO: Load global settings
INFO: Load global settings (done) | time=417ms
INFO: Server id: 147B411E-AZ7iL8YsREWy8I8GwEb0
INFO: Loading required plugins
INFO: Load plugins index
INFO: Load plugins index (done) | time=25ms
INFO: Load/download plugins
INFO: Load/download plugins (done) | time=15ms
INFO: Process project properties
INFO: Process project properties (done) | time=5ms
INFO: Project key: django
INFO: Base dir: /Users/blackcatranch-m3/Documents/GitHub/django
INFO: Working dir: /Users/blackcatranch-m3/Documents/GitHub/django/.sonar
INFO: Load project settings for component key: django
```

Figure 21. Confirmation of completion of scan on Django

```
INFO: Hit the cache for 0 out of 398
INFO: Miss the cache for 398 out of 398: ANALYSIS_MODE_INELIGIBLE [398/398]
INFO: Sensor JavaScript/TypeScript/CSS analysis [javascript] (done) | time=7725ms
INFO: Sensor IaC Kubernetes Sensor [iac]
INFO: There are no files to be analyzed for the Kubernetes language
INFO: Sensor IaC Kubernetes Sensor [iac] (done) | time=90ms
INFO: Sensor TextAndSecretsSensor [text]
INFO: Available processors: 8
INFO: Using 8 threads for analysis.
INFO: Start fetching files for the text and secrets analysis
INFO: Using Git CLI to retrieve dirty files
INFO: Retrieving language associated files and files included via "sonar.text.inclusions" that are tracked by git
INFO: Starting the text and secrets analysis
INFO: 3474 source files to be analyzed for the text and secrets analysis
INFO: 3474/3474 source files have been analyzed for the text and secrets analysis
INFO: Sensor TextAndSecretsSensor [text] (done) | time=1847ms
INFO: ----- Run sensors on project
INFO: Sensor Zero Coverage Sensor
INFO: Sensor Zero Coverage Sensor (done) | time=97ms
INFO: ----- Gather SCA dependencies on project
INFO: Dependency analysis skipped
INFO: SCM Publisher SCM provider for this project is: git
INFO: SCM Publisher 2748 source files to be analyzed
INFO:
INFO: SCM Publisher 2748/2748 source files have been analyzed (done) | time=14109ms
INFO: CPD Executor 339 files had no CPD blocks
INFO: CPD Executor Calculating CPD for 768 files
INFO: CPD Executor CPD calculation finished (done) | time=108ms
INFO: SCM revision ID 'b461519bf5973d7fc149560d2f99acdab71a437d'
INFO: Analysis report generated in 1539ms, dir size=45.4 MB
INFO: Analysis report compressed in 6268ms, zip size=22.1 MB
INFO: Analysis report uploaded in 1614ms
INFO: ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/dashboard?id=django
INFO: Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
INFO: More about the report processing at http://localhost:9000/api/ce/task?id=70d5267d-7585-45df-bbac-2624245c3e96
INFO: Analysis total time: 1:27.976 s
INFO: SonarScanner Engine completed successfully
blackcatranch-m3@JRC-M3 django %
```

Figure 22. SonarQube Dashboard Confirmation

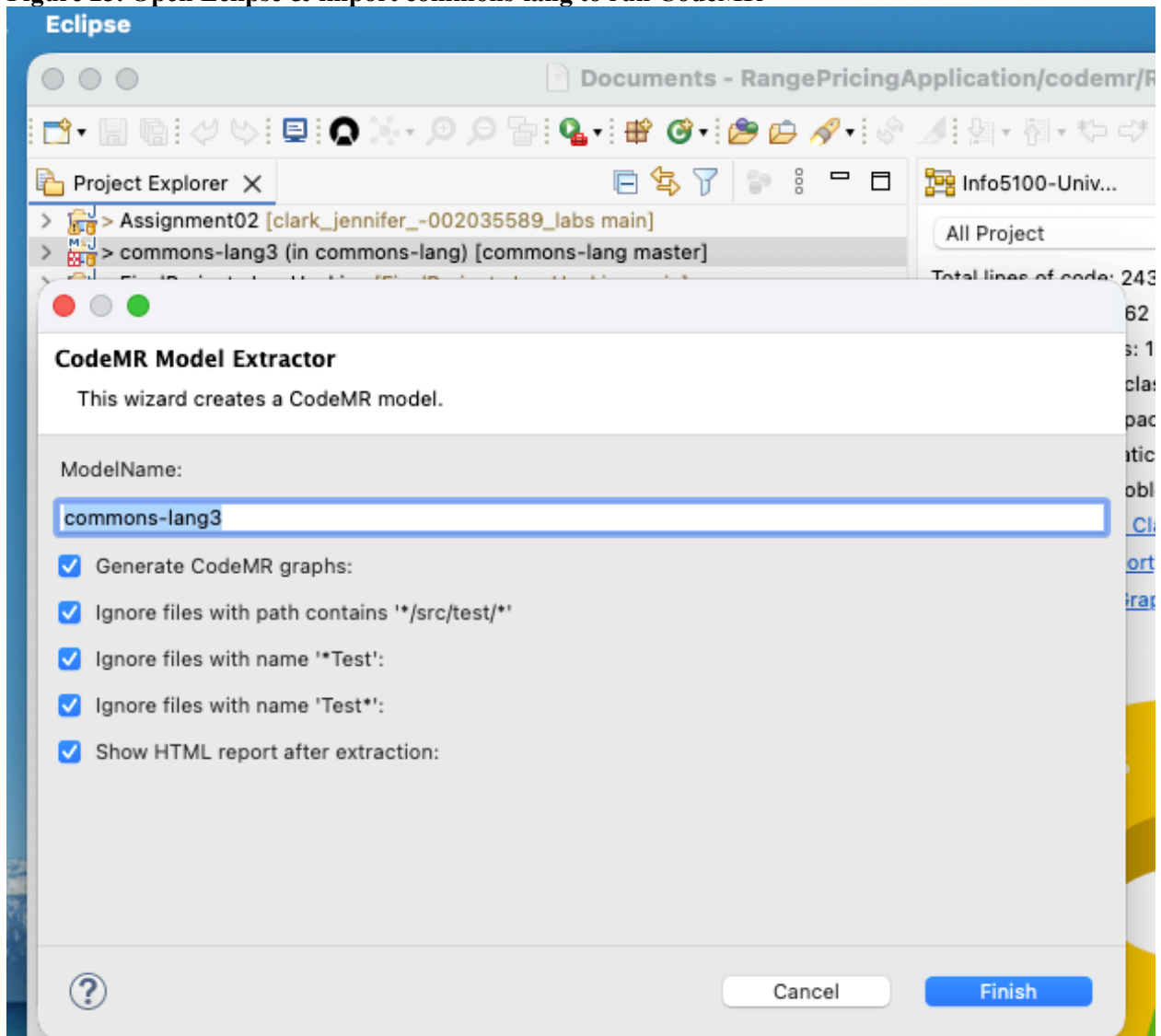
The screenshot shows the SonarQube dashboard for a project named 'Django'. The top navigation bar includes links for Projects, Issues, Rules, Quality profiles, Quality gates, Administration, and More. A notification banner at the top states: 'Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine'. The main content area shows the 'Overview' tab for the 'Django' project, which has 123k Lines of Code and a version not provided. The last analysis was 1 minute ago. The 'Quality Gate' status is 'Passed', indicated by a green checkmark. Below this, there are two tabs: 'New Code' and 'Overall Code'. The 'Overall Code' tab is selected, showing a summary of issues and quality metrics. The metrics are as follows:

Category	Value	Grade
Security	12 Open issues	D
Reliability	69 Open issues	E
Maintainability	1.2k Open issues	A
Accepted issues	0	B
Coverage	0.0%	F
Duplications	0.3%	C
Security Hotspots	74	E

Additional details for Accepted issues: Valid issues that were not fixed. Additional details for Coverage: On 77k lines to cover. Additional details for Duplications: On 176k lines.

APPENDIX 4: CodeMR setup for commons-lang

Figure 23. Open Eclipse & import commons-lang to run CodeMR



APPENDIX 5: Project Setup for spring-framework

Figure 24. Token and Scanner code generation for spring-framework

spring-framework / Analysis Method / Locally

Analyze your project

We initialized your project on SonarQube Community Build, now it's up to you to launch analyses!

1 Provide a token

✔ Analyze "spring-framework" 1: **sqp_93577c310af62b91e302d222423524fa40cc3103**

2 Run analysis on your project

What option best describes your project?

Maven **Gradle** JS/TS & Web .NET Python Other (for Go, PHP, ...)

Execute the Scanner for Gradle

Running an analysis with Gradle is straightforward. You just need to declare the {plugin_code} plugin in your {groovy} or {kotlin} file:

build.gradle build.gradle.kts

```
plugins {  
    id "org.sonarqube" version "7.3.0.8198"  
}
```

Copy

You can find the latest version of the Gradle plugin [here](#).

and run the following command:

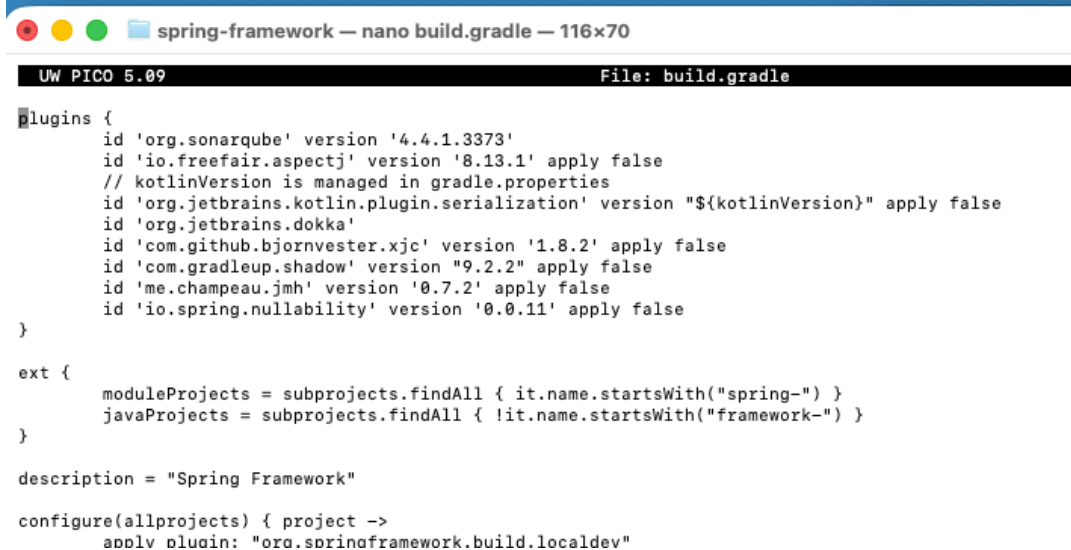
```
./gradlew sonar \  
-Dsonar.projectKey=spring-framework \  
-Dsonar.projectName='spring-framework' \  
-Dsonar.host.url=http://localhost:9000 \  
-Dsonar.token=sqp_93577c310af62b91e302d222423524fa40cc3103
```

Copy

Figure 25. Use Brew to install Gradle

```
[blackcatranch-m3@JRC-M3 spring-framework % brew install gradle
==> Would install 1 formula:
gradle
==> Downloading https://ghcr.io/v2/homebrew/core/gradle/manifests/9.6.0
##### 100.0%
==> Would install 4 dependencies for gradle:
gradle-completion
fontconfig
graphite2
harfbuzz
==> Do you want to proceed with the installation? [y/n]
==> Fetching downloads for: gradle
✓ Bottle fontconfig (2.18.1)                               Preparing 417.0KB/417.0KB
✓ Bottle graphite2 (1.3.15)                               Preparing 92.4KB/92.4KB
✓ Bottle harfbuzz (14.2.1)                                Preparing 3.5MB/3.5MB
✓ Bottle Manifest gradle-completion (9.5.0)               Downloaded 2.0KB/2.0KB
✓ Bottle gradle-completion (9.5.0)                       Downloaded 16.9KB/16.9KB
✓ Bottle gradle (9.6.0)                                   Downloaded 146.6MB/146.6MB
==> Installing dependencies for gradle: gradle-completion, fontconfig, graphite2 and harfbuzz
==> Installing gradle dependency: gradle-completion
==> Pouring gradle-completion--9.5.0.all.bottle.tar.gz
📦 /opt/homebrew/Cellar/gradle-completion/9.5.0: 7 files, 80.2KB
==> Installing gradle dependency: fontconfig
==> Pouring fontconfig--2.18.1.arm64_tahoe.bottle.tar.gz
==> Regenerating font cache, this may take a while
==> /opt/homebrew/Cellar/fontconfig/2.18.1/bin/fc-cache --force --really-force --verbose
█
```

Figure 26. Update build.gradle in spring-framework project to include older version of SonarQube plugin



```
UW PICO 5.09 File: build.gradle
plugins {
    id 'org.sonarqube' version '4.4.1.3373'
    id 'io.freefair.aspectj' version '8.13.1' apply false
    // kotlinVersion is managed in gradle.properties
    id 'org.jetbrains.kotlin.plugin.serialization' version "${kotlinVersion}" apply false
    id 'org.jetbrains.dokka'
    id 'com.github.bjornvester.xjc' version '1.8.2' apply false
    id 'com.gradleup.shadow' version '9.2.2' apply false
    id 'me.champeau.jmh' version '0.7.2' apply false
    id 'io.spring.nullability' version '0.0.11' apply false
}

ext {
    moduleProjects = subprojects.findAll { it.name.startsWith("spring-") }
    javaProjects = subprojects.findAll { !it.name.startsWith("framework-") }
}

description = "Spring Framework"

configure(allprojects) { project ->
    apply plugin: "org.springframework.build.localdev"
```

Figure 27. Troubleshooting compatibility issues with Gradle (with Claude.ai)

```
[blackcatranch-m3@JRC-M3 spring-framework % nano build.gradle
blackcatranch-m3@JRC-M3 spring-framework % ./gradlew sonar \
  -Dsonar.projectKey=spring-framework \
  -Dsonar.projectName='spring-framework' \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.token=sqp_ff5ea4100672fd276bb0638bd9d02af7d5e6abbe \
[ --stacktrace 2>&1 | grep -A 20 "What went wrong" | head -40
]
* What went wrong:
'org.gradle.api.plugins.Convention org.gradle.api.internal.plugins.DslObject.getConvention()'

* Try:
> Run with --info or --debug option to get more log output.
> Get more help at https://help.gradle.org.

* Exception is:
java.lang.NoSuchMethodError: 'org.gradle.api.plugins.Convention org.gradle.api.internal.plugins.DslObject.getConvent
ion()'
    at org.sonarqube.gradle.SonarPropertyComputer.configureSourceDirsAndJavaClasspath(SonarPropertyComputer.java
:315)
    at org.sonarqube.gradle.SonarPropertyComputer.lambda$configureForJava$5(SonarPropertyComputer.java:239)
    at org.gradle.api.internal.collections.CollectionFilter$1.execute(CollectionFilter.java:61)
    at org.gradle.api.internal.DefaultDomainObjectCollection.all(DefaultDomainObjectCollection.java:188)
    at org.gradle.api.internal.DefaultDomainObjectCollection.withType(DefaultDomainObjectCollection.java:230)
    at org.sonarqube.gradle.SonarPropertyComputer.configureForJava(SonarPropertyComputer.java:239)
    at org.sonarqube.gradle.SonarPropertyComputer.addGradleDefaults(SonarPropertyComputer.java:469)
    at org.sonarqube.gradle.SonarPropertyComputer.computeSonarProperties(SonarPropertyComputer.java:103)
    at org.sonarqube.gradle.SonarPropertyComputer.computeSonarProperties(SonarPropertyComputer.java:90)
    at org.sonarqube.gradle.SonarQubePlugin.lambda$configureTask$2(SonarQubePlugin.java:101)
    at org.gradle.api.internal.provider.DefaultProvider.calculateOwnValue(DefaultProvider.java:73)
    at org.gradle.api.internal.provider.AbstractMinimalProvider.calculateValue(AbstractMinimalProvider.java:115)
[blackcatranch-m3@JRC-M3 spring-framework % ./gradlew wrapper --gradle-version 8.12.1
]

[Incubating] Problems report is available at: file:///Users/blackcatranch-m3/Documents/GitHub/spring-framework/build
/reports/problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 10.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your ow
n scripts or plugins.

For more on this, please refer to https://docs.gradle.org/9.6.0/userguide/command_line_interface.html#sec:command_li
ne_warnings in the Gradle documentation.

BUILD SUCCESSFUL in 2s
11 actionable tasks: 1 executed, 10 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.6.0/userguide/configuration_
cache_enabling.html
[blackcatranch-m3@JRC-M3 spring-framework % cat gradle/wrapper/gradle-wrapper.properties
]
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
distributionUrl=https://services.gradle.org/distributions/gradle-8.12.1-bin.zip
networkTimeout=10000
retries=0
retryBackOffMs=500
validateDistributionUrl=true
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
blackcatranch-m3@JRC-M3 spring-framework %
```

Figure 28. Installing older version of Java (21) to help resolve Gradle/SonarQube issue

```
[blackcatranch-m3@JRC-M3 spring-framework % brew install openjdk@21
✓ JSON API packages.arm64_tahoe.jws.json                               Downloaded 15.2MB/ 15.2MB
==> Would install 1 formula:
openjdk@21
==> Downloading https://ghcr.io/v2/homebrew/core/openjdk/21/manifests/21.0.11
##### 100.0%
==> Fetching downloads for: openjdk@21
✓ Bottle openjdk@21 (21.0.11)                                         Downloaded 200.3MB/200.3MB
==> Pouring openjdk@21--21.0.11.arm64_tahoe.bottle.tar.gz
Warning: openjdk@21 was installed but not linked because openjdk is already installed.
To link this version, run:
  brew link openjdk@21
==> Caveats
For the system Java wrappers to find this JDK, symlink it with
  sudo ln -sfn /opt/homebrew/opt/openjdk@21/libexec/openjdk.jdk /Library/Java/JavaVirtualMachines/openjdk-21.jdk

openjdk@21 is keg-only, which means it was not symlinked into /opt/homebrew,
because this is an alternate version of another formula.

If you need to have openjdk@21 first in your PATH, run:
  echo 'export PATH="/opt/homebrew/opt/openjdk@21/bin:$PATH"' >> ~/.zshrc

For compilers to find openjdk@21 you may need to set:
  export CPPFLAGS="-I/opt/homebrew/opt/openjdk@21/include"
==> Summary
📦 /opt/homebrew/Cellar/openjdk@21/21.0.11: 600 files, 347.3MB
==> Running 'brew cleanup openjdk@21'...
Disable this behaviour by setting 'HOMEBREW_NO_INSTALL_CLEANUP=1'.
Hide these hints with 'HOMEBREW_NO_ENV_HINTS=1' (see 'man brew').
blackcatranch-m3@JRC-M3 spring-framework % ]
```

Figure 29. Gradle.properties now points to older version of Java for spring-framework

```
[blackcatranch-m3@JRC-M3 spring-framework % cat gradle.properties
version=7.1.0-SNAPSHOT

org.gradle.caching=true
org.gradle.jvmargs=-Xmx2048m
org.gradle.parallel=true

kotlinVersion=2.3.20
byteBuddyVersion=1.17.6

kotlin.jvm.target.validation.mode=ignore
kotlin.stdlib.default.dependency=false
org.gradle.java.home=/opt/homebrew/opt/openjdk@21
blackcatranch-m3@JRC-M3 spring-framework % ]
```

Figure 30. Down versioning SonarQube for compatibility

```
spring-framework — nano build.gradle — 116x70
UW PICO 5.09 File: build.gradle

plugins {
    id 'org.sonarqube' version '7.3.1.8318'
    id 'io.freefair.aspectj' version '8.13.1' apply false
    // kotlinVersion is managed in gradle.properties
}
```

Figure 31. Terminal confirmation of successful SonarQube build for spring-framework

```
blackcatranch-m3@JRC-M3 spring-framework % grep sonarqube build.gradle
    id 'org.sonarqube' version '5.0.0.4638'
blackcatranch-m3@JRC-M3 spring-framework % nano build.gradle
blackcatranch-m3@JRC-M3 spring-framework % ./gradlew sonar \
-Dsonar.projectKey=spring-framework \
-Dsonar.projectName='spring-framework' \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.token=sqp_ff5ea4100672fd276bb0638bd9d02af7d5e6abbe \
-Dsonar.gradle.skipCompile=true
Use of preview features have been detected during analysis. Enable DEBUG mode to see them.
Use of preview features have been detected during analysis. Enable DEBUG mode to see them.

[Incubating] Problems report is available at: file:///Users/blackcatranch-m3/Documents/GitHub/spring-framework/build/reports/problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 10.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plugins.

For more on this, please refer to https://docs.gradle.org/9.6.0/userguide/command_line_interface.html#sec:command_line_warnings in the Gradle documentation.

BUILD SUCCESSFUL in 11m 39s
39 actionable tasks: 29 executed, 10 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.6.0/userguide/configuration_cache_enabling.html
blackcatranch-m3@JRC-M3 spring-framework %
```

Figure 32. Localhost web SonarQube dashboard for spring-framework

localhost:9000/dashboard?id=spring-framework&codeSco...

Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine

SonarQube community

Projects Issues Rules Quality profiles Quality gates Administration More

spring-framework / Overview

Overview

410k Lines of Code • Version 7.1.0-SNAPSHOT • Last analysis 12 minutes ago

Quality Gate **Passed**

New Code Overall Code

Security 46 Open issues	Reliability 858 Open issues	Maintainability 16k Open issues
Accepted issues 0 Valid issues that were not fixed	Coverage 0.0% On 191k lines to cover.	Duplications 4.1% On 866k lines.
Security Hotspots 25		

APPENDIX 6: SonarQube Comparative Summaries

Table 1: SonarQube Results Overview

Metric	Django (Python)	commons-lang (Java)	spring-framework (Java)
Lines of Code	123k	33k	410k
Quality Gate	Passed	Passed	Passed
Security	12 (D)	0 (A)	46 (E)
Reliability	69 (E)	47 (C)	858 (E)
Maintainability	1.2k (A)	990 (A)	16k (A)
Security Hotspots	74	11	25
Duplications	0.3%	1.3%	4.1%

APPENDIX 7: CodeMR Report Comparative Summaries

Table 2: Project Metrics Summary

	commons-lang	JavaHuskies_Final
LOC	23,329*	9317
CLASSES	374	89
PACKAGES	18	9
EXTERNAL PACKAGES	25	56
EXTERNAL CLASSES	1,402	588
PROB CLASSES	46	23
HI PROB CLASSES	0	0

* CodeMR shows 23K LOC compared to SonarQube's 33K due to different methods of counting lines (all lines vs. NCLOC).

Table 3: Project "C" Metrics Summary

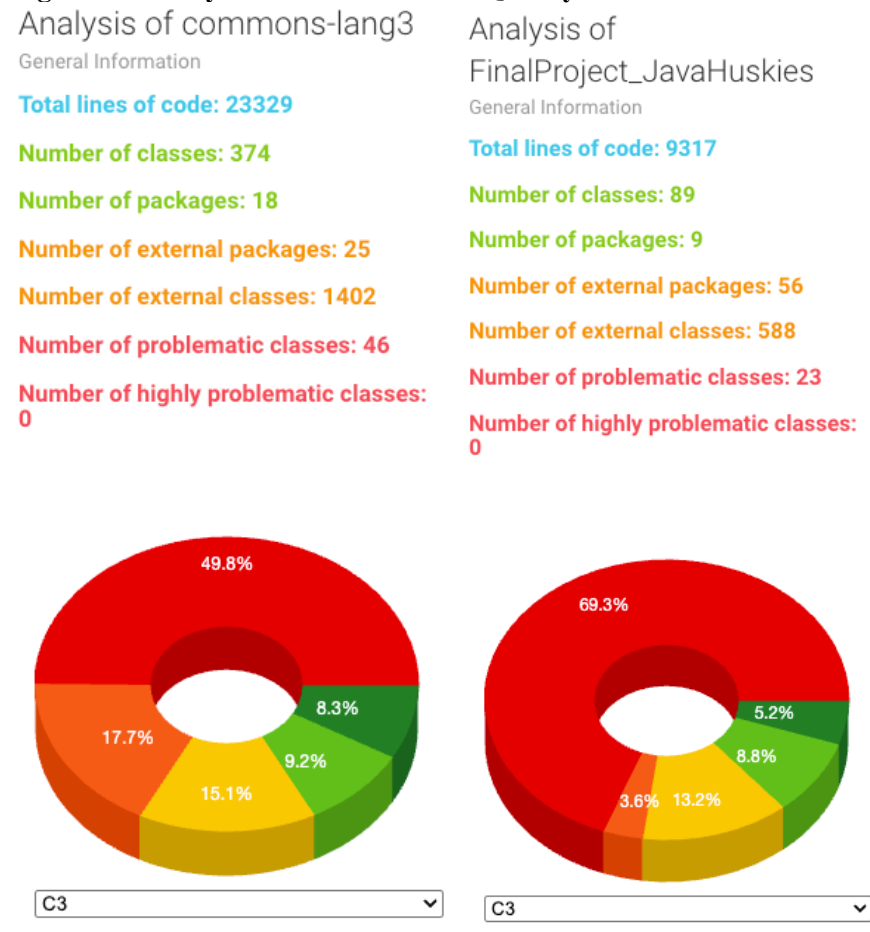
	commons-lang	JavaHuskies_Final
C3	49.8% very high, 17.7% high, 15.1% med-high	69.3% high
Complexity	47.3% very high, 10.1% high	69.3% high
Coupling	68.6% low, 12.8% low-medium	65.9% low/med-low
Cohesion	28.7% high, 20.5% very high, 21.8% med-high	55.4% low/med-low

APPENDIX 8: CodeMR Report Visuals Comparison

CodeMR analysis of **commons-lang** identified 46 problematic classes out of 374 (12.3%) with zero highly problematic classes. C3 complexity skewed high (49.8% very high) but coupling was strong at 68.6% low, meaning complex individual methods within well-isolated classes. Package dependencies showed well-distributed inter-package relationships, contrasting with the red-heavy treemap.

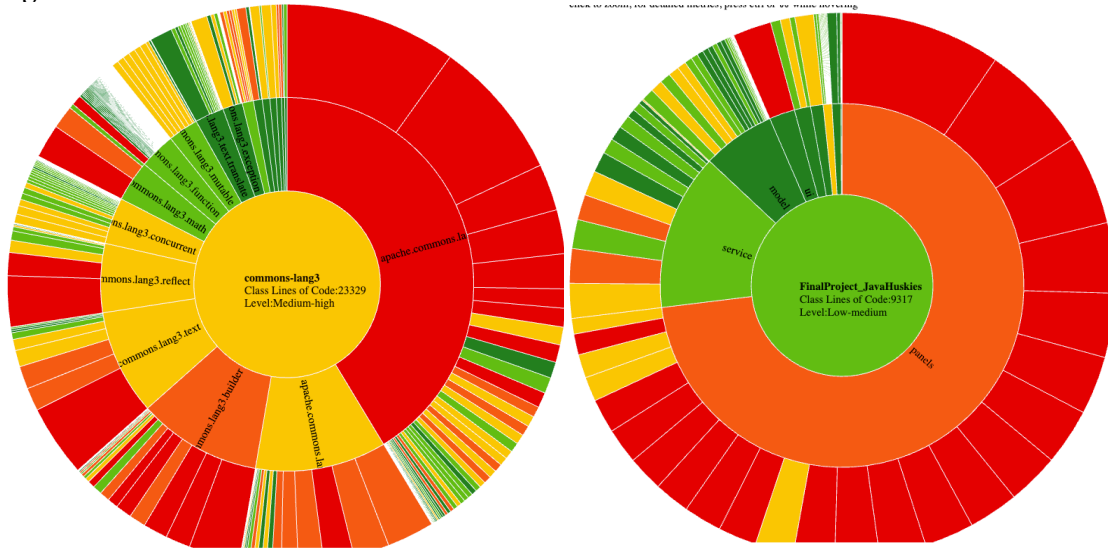
commons-lang3 is approximately 2.5 times larger than **FinalProject_JavaHuskies** by LOC (23,329 vs. 9,317), yet both projects show nearly equivalent combined percentages of Very High and High C3 complexity, suggesting that commons-lang carries proportionally less complexity per class despite its scale. Neither codebase has any highly problematic classes.

Figure 33. Analysis & Distribution of Quality Attributes



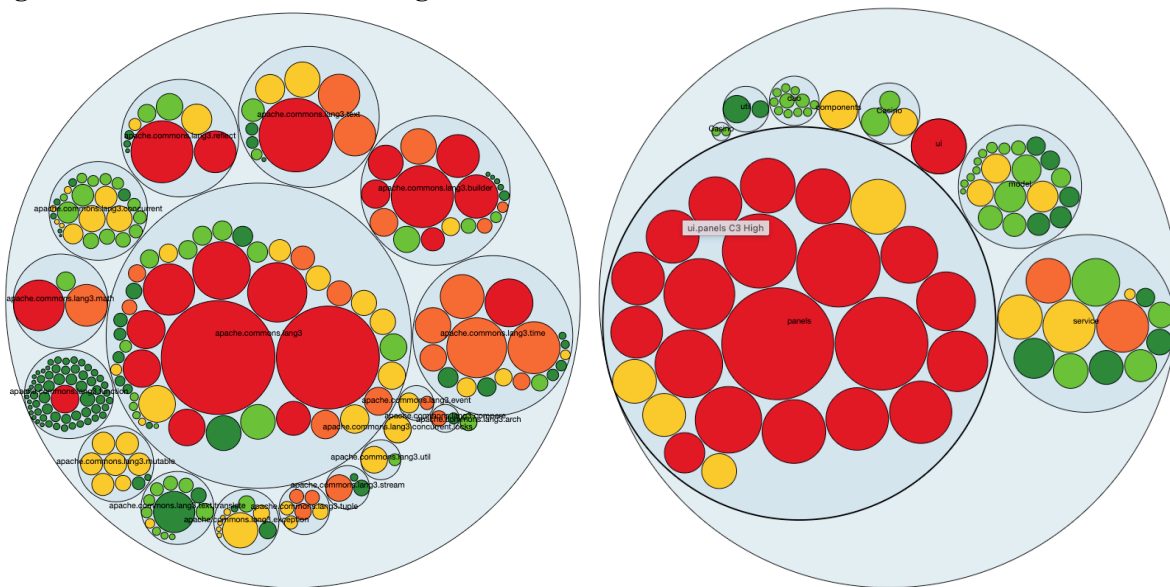
Note: commons-lang is on the left and FinalProject_JavaHuskies on the right for each area in the figures below:

Figure 34. Metric Values in Sunburst Chart



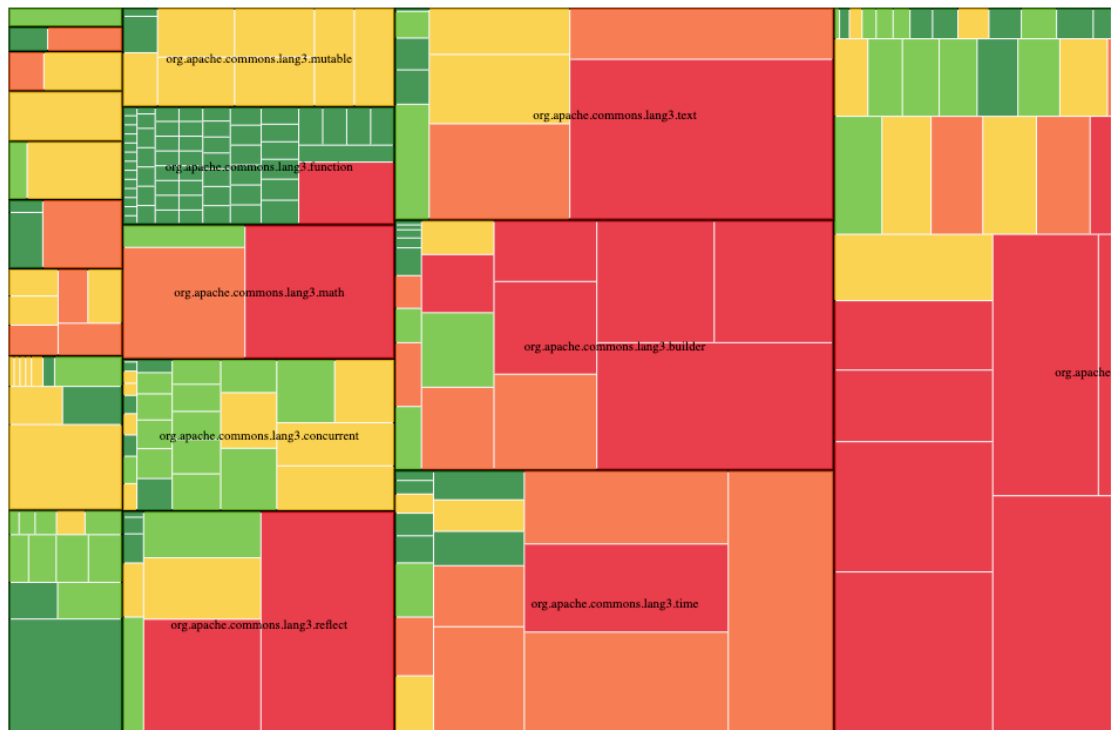
The Sunburst Charts (Figure 34) reveal different structural signatures: commons-lang shows significant red at the outer rings with yellow concentrated toward the center, indicating that complexity is distributed across many mid-sized classes. FinalProject_JavaHuskies presents a green center with denser orange radiating outward, a pattern consistent with a smaller number of large, complex classes dominating the profile.

Figure 35. Metric Values in Package

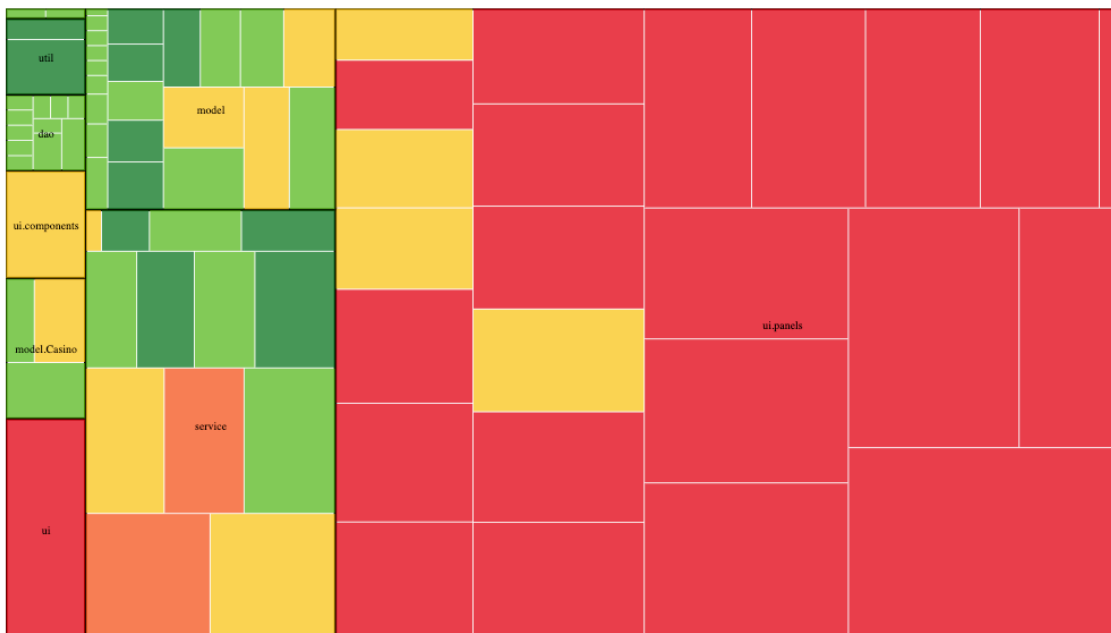


The Metric Values by Package bubble charts (Figure 35) reinforce this distinction. commons-lang's complexity is distributed across its largest bubbles and several mid-sized ones, with reds and oranges spread across multiple packages. FinalProject_JavaHuskies shows red concentrated in fewer, larger bubbles, indicating over-concentration of complexity in a small number of packages.

Figure 36. Metric Values in Treemap Chart
Commons-lang

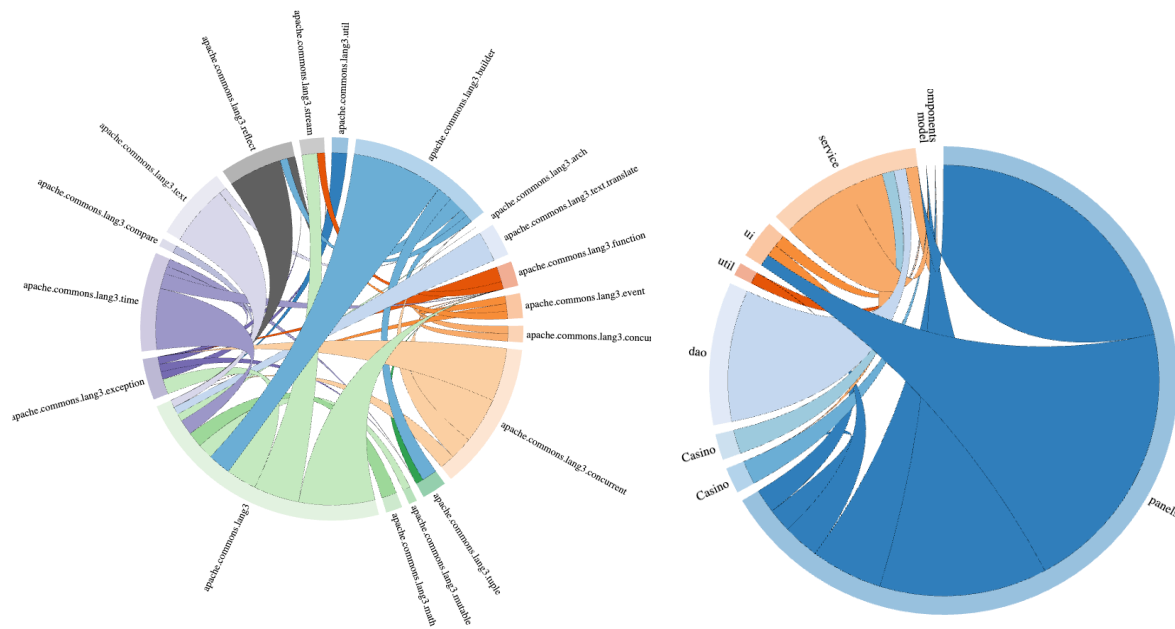


JavaHuskies Final



The Treemap Charts (Figure 36) tell a similar story. FinalProject_JavaHuskies is predominantly solid red, indicating high complexity concentrated across most of its classes. commons-lang's treemap is also weighted toward red from the center rightward, but includes substantial orange, yellow, and green across the full width -- a more varied and distributed complexity profile.

Figure 37. Package Dependencies



The Package Dependencies chord charts (Figure 37) show the most striking contrast. FinalProject_JavaHuskies is nearly solid blue, indicating an overwhelming dependency concentration on the "panels" package -- a structural risk identified in the M06 analysis. commons-lang, by contrast, shows well-distributed inter-package dependencies with no single dominant color, reflecting balanced architectural integration across all 18 packages.

This difference may be attributable in part to the development context: commons-lang is a mature open-source library developed collaboratively by multiple contributors over many years, while FinalProject_JavaHuskies was architected from a single AI-generated framework and built out from that initial structure.

REFERENCES

Apache Software Foundation. (n.d.). commons-lang [Source code repository]. GitHub. Retrieved on June 18, 2026 from <https://github.com/apache/commons-lang>

CVE Details. (n.d.). Django vulnerability list. Retrieved on June 20, 2026 from https://www.cvedetails.com/vulnerability-list/vendor_id-10199/product_id-18211/Djangoproject-Django.html

Django Software Foundation. (n.d.). django [Source code repository]. GitHub. Retrieved on June 18, 2026 from <https://github.com/django/django>

OpenLogic. (n.d.). Managing EOL Spring vulnerabilities. OpenLogic by Perforce. Retrieved on June 20, 2026 from <https://www.openlogic.com/blog/managing-eol-spring-vulnerabilities>

VMware, Inc. (n.d.). spring-framework [Source code repository]. GitHub. Retrieved on June 18, 2026 from <https://github.com/spring-projects/spring-framework>

Wikipedia. (2022, February 2). Log4Shell. Wikipedia. Retrieved on June 20, 2026 from <https://en.wikipedia.org/wiki/Log4Shell>

Methodology and Use of AI Tools

Claude was used to troubleshoot the Gradle project and evaluate my draft against the rubric. The analysis, observations, and conclusions reflect the author's own hands-on experience with the tool and the codebases analyzed. All content has been reviewed for accuracy and represents the author's understanding of the results.

Reference to Claude LLM conversation on Gradle SonarQube plugin troubleshooting:

<https://claude.ai/share/9cab08bb-a793-4de8-9b8b-fdc809956f68>

Reference to Claude LLM conversation on rubric compliance:

<https://claude.ai/share/b6e2c24e-f1eb-463f-9132-3a545296cd00>